AI Foundations: VS Code + R

Nicholas Warren

Lead, Artificial Intelligence & Machine Learning program, DISSC

nick.warren@yale.edu

Maurice Dalton

Lead, Data Engineering and Solution Team, DISSC

Prerequisites

Complete the setup instructions before starting.

Goals

- Learn how Claude Code can accelerate your research workflow in R
- Practice using AI to explain, iterate on, and extend a real statistical analysis
- Understand the end-to-end setup so you can use these tools independently after the workshop

Why this matters

Research workflows in R involve repetitive, time-consuming tasks: cleaning data, debugging cryptic error messages, writing boilerplate code, and iterating on visualizations. Claude Code can handle these so you can focus on the research questions.

- **Hours to minutes.** Tasks like reformatting data, writing plot code, or adding robust standard errors can be done in a single prompt.
- **Lower the barrier.** You don't need to memorize package syntax. Describe what you want in plain English and iterate from there.
- **Stay in your editor.** No context-switching to a browser. Claude works alongside your code and can read your files directly.

Workshop outline

Phase	Description	Duration
Setup	Verify installations and configure tools	~15 min
Overview	Terminal, VS Code, Claude Code, memory, skills	~20 min
1. Idea generation	Explore the research question	~5 min
2. Script creation	Import data and build analysis code	~5 min
3. Analysis & visualization	Iterate on plots, scales, and models	~10 min
4. Debugging	Diagnose and fix broken code	~5 min

What is the terminal?

The **terminal** is a text-based interface for your computer. Instead of clicking icons, you type commands. Think of it like the R console, but for your entire machine.

Concept	What it means
Shell	The program that runs your commands (`zsh` , `bash` , `cmd`)
Command	A text instruction like `ls` (list files) or `cd` (change folder)
Working directory	The folder your terminal is currently "in"

Essential terminal commands

These are the commands you'll use most often:

Command	What it does	Example
`ls`	List files in the current folder	`ls data/`
`cd`	Change directory (move to another folder)	`cd ~/Documents`
`pwd`	Print the current working directory	`pwd`
`cat`	Print the contents of a file	`cat data.csv`
`claude`	Start Claude Code	`claude`

> You don't need to memorize these. Claude Code can run them for you.

Key concepts from setup

Tool	What it is
Homebrew	A macOS package manager. Like `install.packages()` but for any software.
Quarto	A publishing tool. `.qmd` files mix code and text, like R Markdown.
AWS Bedrock	Amazon's managed AI service. Your prompts route through Yale's AWS account.
API key	A password-like token that authenticates your requests to the AI service.

What is VS Code?

Visual Studio Code (VS Code) is a free, open-source code editor made by Microsoft. It works on macOS, Windows, and Linux. Think of it as a modern replacement for RStudio or whatever editor you currently use for writing code.

Key differences from RStudio:

- VS Code supports **any programming language** (R, Python, Stata, etc.) through extensions
- It has a built-in **terminal** where you can run shell commands
- It is the most widely used code editor in the world, with a large ecosystem of extensions

Navigating the VS Code interface

Area	Location	What it does
Activity Bar	Far left (icons)	Switch between Explorer, Search, Extensions
Side Bar	Left panel	File tree, search results, or extensions
Editor	Center	View and edit files. Run code with `Cmd/Ctrl + Enter`
Panel	Bottom	Terminal and R Terminal. Toggle with `Ctrl + ` `

Key actions in VS Code

Action	How
Find your files	Click the top Activity Bar icon to open the **Explorer**
Open a terminal	`Ctrl + ` ` or `Terminal > New Terminal`
Install extensions	`Cmd/Ctrl + Shift + X` to open the Extensions view
Command Palette	`Cmd/Ctrl + Shift + P` — universal search for any command or setting

How it works

In this workshop you use **VS Code** as your editor with the **R** and **Quarto** extensions, and **Claude Code** as your AI assistant. Claude Code is a command-line tool made by Anthropic that brings Claude directly into your terminal.

Your prompts and code are sent to **Claude Opus 4.6** through **AWS Bedrock**, Amazon's managed AI service, running on Yale's AWS account.

VS Code for RStudio users

RStudio	VS Code equivalent
Console	**R Terminal** (bottom Panel)
Source editor	**Editor** (center). Run lines with `Cmd/Ctrl + Enter`
Files pane	**Explorer** (top icon in Activity Bar)
Environment pane	**R: Workspace** (R icon in Activity Bar)
Plots pane	Inline in the Quarto notebook
Help	Hover over a function, or `?fn` in R Terminal

What is Claude Code?

Claude Code is Anthropic's official command-line tool that brings Claude directly into your terminal. Unlike ChatGPT or other web-based assistants, Claude Code runs where you work (inside VS Code's terminal) and can see your project files.

It can:

- **Read and understand your project files** without copying and pasting
- **Edit files directly** when you ask it to make changes
- **Run commands** like R scripts, git operations, and more
- **Answer questions** about your code, data, and analysis

Data security

- **Your data is not used for training.** AWS Bedrock does not use your prompts or outputs to train any models.
- **Your data is not shared.** Model providers (including Anthropic) do not have access to your inputs, outputs, or logs.
- **Enterprise compliance.** AWS Bedrock is HIPAA eligible, SOC and ISO certified, and GDPR compliant.

In short: you get the benefits of a frontier LLM without research data leaving Yale's AWS environment or being used to improve future models.

Open the workshop notebook

- In the **Explorer** (top icon in the Activity Bar on the left), navigate to `files > code` and click `example.qmd`
- This is a **Quarto notebook** that mixes markdown text with R code chunks (similar to R Markdown)
- Click the **Run Cell** button (play icon) above any code chunk to run it
- Output (plots, tables, text) appears inline below each chunk

Start Claude Code

Open a **new terminal** in VS Code: press `Ctrl + ` ` or go to `Terminal > New Terminal` in the menu bar. In the terminal that appears at the bottom, type:

```
claude
```

This opens an interactive session. Useful commands:

Command	Action	Command	Action
`/help`	Show commands	`/model`	Switch models
`/clear`	Clear history	`Ctrl + C`	Cancel response
`/exit`	Exit Claude Code	`Ctrl + D`	Exit Claude Code

Select the right model

Once Claude Code is running, type `/model` and select **Opus 4.6** from the list. This is the most capable model and the one we'll use throughout the workshop.

> If you skip this step, Claude Code may default to a less capable model that produces lower-quality results.

Two terminals, two tools

VS Code's bottom Panel can have multiple terminals open at once. In this workshop you'll use two:

- **R Terminal**: runs R code. Use it for `install.packages()`, `library()`, and running code chunks. Click `R: (not attached)` in the status bar to start it.
- **Terminal** (zsh / bash / cmd): a regular shell. This is where you run `claude` to start Claude Code.

To switch between them, click the **dropdown arrow** next to the terminal name in the Panel header (e.g., it might say "R Interactive" or "zsh"). You can also click the **+** icon to create a new terminal.

Tips for working with Claude Code

- **Be specific.** "Add clustered standard errors at the state level" works better than "improve the model."
- **Point to files.** Claude Code can read your project, but referencing a file by name (e.g., "in example.qmd") focuses its attention.
- **Iterate.** If the first result isn't right, follow up: "That's close, but use `sandwich` instead of `estimatr`."
- **Ask it to explain.** "Why did you use that approach?" is always a good follow-up.

Review before you accept

Claude Code will ask for permission before editing files or running commands. Before approving:

- **Read the diff.** Claude Code shows you exactly what it plans to change. Make sure you understand the edit before accepting.
- **Run the code yourself.** After Claude edits a chunk, run it to confirm it produces the output you expect.
- **Verify statistical claims.** AI can make reasoning errors. Check coefficients, p-values, and interpretations against your own understanding.

Claude Memory

Claude Code can **remember things across sessions**. When you correct it, share a preference, or teach it something about your project, it can save that as a memory and recall it next time.

- **User memories:** your role, preferences, and expertise level
- **Project memories:** context about your codebase and ongoing work
- **Feedback memories:** corrections you've given (e.g., "don't use that package")

Type `/memory` in Claude Code to see what it has stored and toggle memory on or off.

Where memory lives

Memory files are plain markdown stored on your machine. You can read, edit, or delete them at any time. They are never uploaded or shared.

Location	What it contains
`~/.claude/CLAUDE.md`	Your instructions for all projects
`.claude/CLAUDE.md`	Project-specific instructions (shareable via git)
`~/.claude/projects/<name>/memory/`	Notes Claude writes itself from your sessions

Skills

Skills are reusable prompt templates that work like custom slash commands.

Skill	What it does
`/memory`	View and manage memory files
`/model`	Switch between available models
`/simplify`	Review changed code for quality and efficiency
`/compact`	Summarize the conversation to free up context
`/init`	Generate a starting `CLAUDE.md` for your project

Custom skills

You can create your own skills by adding a `SKILL.md` file to your project:

- **Personal:** `~/.claude/skills/<name>/SKILL.md` (all your projects)
- **Project:** `.claude/skills/<name>/SKILL.md` (shared via git)

This workshop includes a `/modernize` skill that converts old R scripts to tidyverse style. Open `.claude/skills/modernize/SKILL.md` in the Explorer to see how it works, then try it: type `/modernize` in Claude Code.

> Full docs: https://code.claude.com/docs/en/skills

Phase 1: Concept & Idea Generation

Use Claude Code to explore the research question **before writing any code**.

Try these prompts:

> What is Difference-in-Differences and when is it appropriate to use?

> I have organ donation rate data for 27 states across 6 quarters. California introduced an "active choice" policy in Q3 2011. How would I set up a DiD analysis?

Phase 2: Script Creation

Ask Claude Code to write the import code. It can edit the notebook directly.

> Read in data/organ_donations.csv and prepare it for a DiD analysis. Write the code in the Phase 2 section of files/code/example.qmd.

Run the chunks Claude creates to verify they work.

Phase 2: About the data

The dataset (`data/organ_donations.csv`) has 162 rows: 27 states across 6 quarters (Q4 2010 through Q1 2012). California's "active choice" organ donation policy took effect in Q3 2011.

Data source: Kessler & Roth (2014), "Don't Take 'No' For An Answer." Dataset via the `causaldata` R package, featured in *The Effect* by Nick Huntington-Klein.

Phase 3: Analysis & Visualization

The notebook has pre-written code for a DiD plot and regression model. Run these chunks, then use Claude Code to iterate:

> Change the y-axis to show values from 0% to 100%

> Switch the plot to a black-and-white theme suitable for print

> Add robust standard errors clustered at the state level

> Create a summary statistics table broken down by treatment group and period

Phase 3: More ideas

> Export the coefficient table as a CSV and the plot as a high-resolution PNG to an output/ folder

> Add a placebo test using a fake treatment date one quarter earlier

> Explain the coefficients from the DiD regression in plain language

See `files/prompts/examples.md` for more prompts.

Phase 4: Debugging

The bottom of the notebook has four code chunks with intentional bugs. Ask Claude Code to find and fix them:

> Look at the Phase 4 section of files/code/example.qmd and fix the bugs

Then try a follow-up:

> For each bug you fixed, explain what went wrong and how a beginner could spot the same mistake

Troubleshooting

- **Claude Code not responding:** Restart your terminal and verify credentials (Step 5 in setup). Run `echo $CLAUDE_CODE_USE_BEDROCK` to confirm it's `1`.
- **R packages not found:** Run `install.packages("tidyverse")` in the R Terminal.
- **No code completion:** Make sure `languageserver` is installed (Step 12 in setup). Restart VS Code.
- **Script errors:** Check the working directory with `getwd()` in the R Terminal.

Next steps

- **Get long-term access.** Your workshop API key will expire. Contact dissc@yale.edu for ongoing access.
- **Try your own data.** Bring a dataset and a research question. Claude Code can help you write and refine the analysis.
- **Explore Claude Code further.** Try asking it to create new scripts, debug errors, or explain unfamiliar code.
- **Get in touch.** nick.warren@yale.edu or dissc@yale.edu